Deploying the AIMScribe Server to DigitalOcean

The complete manual: accounts, droplet, storage, the stack, the endpoints, the clinic PCs, and keeping it running · AIMS LAB · 23 October 2026 (server 3.7, recorder 3.7.0)

This manual moves the **whole AIMScribe v3 server** — nine services, both databases, the gateway, the archive worker, the backups and the monitor — onto one DigitalOcean droplet in Bangalore, with the audio in Cloudflare R2, **and keeps it up to date by itself**: whatever is pushed to GitHub's `main` is deployed within minutes, checked, and rolled back if it is not healthy. It is written for someone who has never run a Linux server: every step says **what to type**, **what you should see**, and **what to do if you see something else**.

	Stage	Time	Who
0	**Accounts** — DigitalOcean and Cloudflare, owned by one email, with two-factor login	30 min	the owner
A	**The endpoint status table** — what works before and after. Give this to CMED	read it now	you → CMED
B	Before you start: the checklist, the three R2 buckets, the plan	45 min	AIMS LAB
C	**The deployment, thirteen steps** — the last one turns on automatic updates	**3–4 hours**	AIMS LAB
D	Proving every endpoint answers	20 min	AIMS LAB
E	**Daily and weekly checks** — one page	5 min a day	AIMS LAB
F	Updates, rollback, **who may send audio**, go-live checklist	30 min	AIMS LAB

The one thing to understand before starting

CMED's page never learns the server address. It talks to the recorder on the same PC, over loopback. Only **CMED's own server** and **each recorder** hold the AIMS LAB address. So this deployment changes exactly two things on the outside: one setting in CMED's server, and the server address in each recorder (set by re-running the installer, step 11). CMED's page is not touched.

How to read the boxes in this manual

Grey box, typewriter text — type it exactly. Lines starting with # are explanations; you may type them or leave them out.
<angle brackets> — replace with your own value, without the brackets.
Green text — what a correct result looks like.
"If you see this instead" tables — the common failures, and the fix for each.
Commands that start with PS> are typed in **Windows PowerShell on your own laptop**. Everything else is typed **on the droplet**, over SSH.

What it runs on: the specification

Item	Specification	About, a month
Droplet	`s-8vcpu-16gb`: 8 vCPU, 16 GB memory, 320 GB SSD, **Bangalore (BLR1)**, Ubuntu 24.04 LTS. Step 1	$96
Droplet backups	Weekly, kept by DigitalOcean. Step 1	$19
Data volume	100 GB block storage, encrypted at rest, mounted on `/srv/aimscribe`: the databases, a week of nightly backups, and the working space where a confirmed recording is joined before it goes to Cloudflare. **No audio is kept here.** Step 7	$10
Cloudflare R2	Three private buckets: segments, **archive**, backups. 3 TB of archive at $15 a TB, no charge for downloads. Section 6	$45
Domain	One name pointing at the droplet, for the certificate. Section 4	UIU / registrar
Total	Grows with the archive by about $15 for each further TB	**about $173**

Sized for	Seven clinics, about 1,000 consultations a day
Audio	WAV, 44.1 kHz: about 318 MB an hour of recording
Database	PostgreSQL 16 in 6 GB (`postgresql.16gb.conf`), pooled by pgbouncer. A year of recordings is a few hundred thousand rows
Open to the internet	Ports 80 and 443 only, through the gateway; SSH from your own address only
Updates	Pulled from GitHub `main` every two minutes; built, migrated, health-checked, rolled back on failure. Step 13
Where the archive lives	**Cloudflare R2 only**, encrypted before upload. A WAV leaves the droplet within 10 minutes of its copy being proven. Section 6b

Why 100 GB is enough: no audio is stored on DigitalOcean

On the volume	Size	Why
Both databases	**about 3 GB** for 18,000 recordings; **50 GB** allowed	Measured rows: about 130 KB a recording with indexes (session, ~60 clips, the signed chain, receipts, audit) = 2.3 GB; the clinical record ~25 KB a visit = 0.5 GB. PostgreSQL needs about twice its data for its write log, clean-up and index rebuilds; the rest is years of growth
Working space	**under 5 GB**; 20 GB allowed	A confirmed recording's pieces are joined (in memory), written, encrypted and copied to Cloudflare, and the WAV deleted within 10 minutes of the copy being proven (`AIMS_ARCHIVE_LOCAL_RETENTION_DAYS=0`). A recording CMED has not confirmed stays as its pieces in Cloudflare, not here
Nightly backups	**about 20 GB**	7 nights here, encrypted; 90 in Cloudflare (`aimscribe-backups`)
Total	**about 90 GB**	A 100 GB volume; it can be enlarged later without stopping the server

The droplet's own 320 GB disk holds Ubuntu (20 GB), Docker's images and build cache (25 GB), logs (10 GB) and swap (4 GB): about 60 GB.

PART 0

Accounts

Two accounts, one owner email, two-factor login on both

0.1 · The owner email

Every account and every automatic notice in this deployment goes to one address, so nothing important lands in a mailbox nobody reads:

Where the email is used	Why
DigitalOcean account owner	billing, droplet alerts, password resets
Cloudflare account owner	R2 storage, billing, API token notices
`AIMS_ACME_EMAIL` **in the server settings (step 6)**	Let's Encrypt writes here if the HTTPS certificate is about to expire
DigitalOcean monitoring and uptime alerts (step 12)	CPU, memory, disk, and "the server stopped answering"
The comment on your SSH key (step 1)	so you can tell whose key it is later

```
Owner email:   your-team-email@example.org
```

The server's own alerts do not go to email. The monitor service sends its alerts (disk filling, recordings not confirmed, backups missing) to a **webhook** — `AIMS_ALERT_WEBHOOK` — such as a Slack, Discord or Google Chat incoming-webhook address. There is no email setting in the stack. Email alerts come from DigitalOcean's own monitoring, set up in step 12.

0.2 · DigitalOcean

	Do this	Where
1	Sign up with **your-team-email@example.org**. Add a payment card	cloud.digitalocean.com → Sign up
2	**Turn on two-factor login** (an authenticator app, not SMS)	avatar → My Account → Security → Two-factor authentication
3	Create a team named **AIMS LAB**; invite each lab member who will administer the server, as **Member** (not Owner)	Settings → Team → Invite members
4	Set a **billing alert** at $150 a month, so a mistake shows before the invoice does	Billing → Billing alerts
5	Create a project named **AIMScribe**; everything in this manual goes in it	left menu → New Project

If you see this instead	What it means, and what to do
"Account under review" after sign-up	DigitalOcean checks new accounts. It can take a few hours. Nothing else in this manual can start until it clears.
The card is declined	Some Bangladeshi cards need international online payments enabled by the bank first. Or use PayPal in the same billing page.

0.3 · Cloudflare

	Do this	Where
1	Sign up with **your-team-email@example.org**	dash.cloudflare.com → Sign up
2	**Turn on two-factor login**	My Profile → Authentication
3	Open **R2 Object Storage** and add a payment method. R2 has a free tier, but a card is required to enable it	left menu → R2 Object Storage
4	Note your **Account ID**. It is part of the storage address	R2 Object Storage overview page, right-hand side
5	Invite lab members who need it, with the narrowest role that works	Manage Account → Members

Write the recovery codes down
Both services give **recovery codes** when two-factor login is turned on. Store them with the access sheet, off this laptop. Losing the phone without them can lock AIMS LAB out of its own server and its own recordings for days.

PART A

The endpoint status table

What works today, what works after deployment - the page to send CMED

1 · Every endpoint, before and after

This is the table to put in front of CMED. "Today" means the system as it runs now, on a PC on a private network. "After" means once Part C is done.

	Endpoint	Today	After	Why
1	`GET https://<host>/health`	**NO**	**YES**	Needs a public name and a certificate. Both arrive with the droplet. No key
2	`GET https://<host>/`	**NO**	**YES**	same
3	`POST https://<host>/fhir/r4/$process-message` *API 2 (after a token from /api/v2/oauth/token)*	**NO**	**YES**	**The one CMED is waiting for.** Finished and tested; it has had no public address to answer on. A FHIR message with a 5-minute token for one hospital. No token: **401**
4	`POST https://<host>/fhir/r4/$process-message` *API 3, Channel B*	**NO**	**YES**	same as 3
5	`ws://127.0.0.1:5050/ws` — `start` *API 1*	**YES**	**YES**	**Unaffected.** Loopback on the clinic PC — never involved our server
6	`ws://127.0.0.1:5050/ws` — `prescription_built` *API 3, Channel A*	**YES**	**YES**	Unaffected
7	`ws://127.0.0.1:5050/ws` — `doctors`	**YES**	**YES**	Unaffected
8	`ws://127.0.0.1:5050/ws` — `status`	**YES**	**YES**	Unaffected
9	`GET https://<host>/fhir/r4/Encounter?patient.identifier=<id>` *what we hold*	**NO**	**YES**	same as 3. No token: **401** `INVALID_TOKEN`
10	`POST https://<host>/api/v2/session/reassign` *new: recorders only*	—	**YES**	Not CMED's. Needs schema script 013. No device token: **401**
11	`POST https://<host>/api/v2/device/health` *new: recorders only*	—	**YES**	Not CMED's. Needs schema script 014. No device token: **401**

Read the shape of that table. The four local ones already work and are untouched by the deployment. The five server ones CMED calls are blocked by one cause — no public address — and are unblocked by one act. Rows 10 and 11 are the recorders' own, and need nothing from CMED.

2 · What API 2 and API 3 Channel B answer

The replies are the same before and after deployment. These are the ones CMED's outbox must act on, including what happens when it sends the same visit twice:

CMED sends	Reply	What it means / what CMED does
A new visit (API 2) or prescription (API 3b)	**202** `ACCEPTED`	Stored. Remove it from the outbox. For API 2, `previous_visit.status` says whether the history was `ALREADY_STORED`, `UPDATED` or `STORED`
The identical body again (a retry after a timeout, or after an outage)	**200** `ALREADY_RECEIVED`	**Nothing new stored.** It was already here. Remove it from the outbox
API 2 for the same visit (same five fields) with a different body	**202** `ACCEPTED`	Stored as a newer record of that visit; **the newest is the one used for matching**. Remove it from the outbox
A prescription for the same visit with a changed body	**202** `ACCEPTED`, `"version": 2`	Kept as version 2 — a correction. Earlier versions are kept too
The same prescription rebuilt — a new `issued_at`, keys reordered	**200** `ALREADY_RECEIVED`	Same fingerprint, so nothing new. Remove it from the outbox
No key, or a wrong one	**401**	Do not retry. Check the key
Body over 1 MB	**413**	Do not retry. Report it to AIMS LAB
A field malformed, or a previous visit named by only some of its five fields	**422** `SCHEMA_INVALID`	Kept aside for review. Do not retry the same body; fix and send again
Our server down, the branch's internet down, or a network error	no reply / **5xx**	**Keep it in the outbox and retry later.** Late is fine: it attaches to its recording whenever it lands

The five fields are `patient_id`, `doctor_id`, `hospital_id`, `start_time` and `date`. The full rules are in CMED_1, the software integration document.

3 · What to say to CMED, in writing

A note you can send as it stands:

```
The four endpoints your page uses on the clinic PC work today and are not
affected by our deployment. Your front-end team can build and test against
them now - install the recorder on one PC and they are live:

    ws://127.0.0.1:5050/ws   commands: start, prescription_built,
                                       doctors, status

The endpoints your server uses are finished and tested, and will answer
as soon as our DigitalOcean deployment is done:

    POST  https://<host>/api/v2/oauth/token        - a 5-minute token per hospital
    POST  https://<host>/fhir/r4/$process-message  (API 2 and API 3, Channel B)
    GET   https://<host>/fhir/r4/Encounter?patient.identifier=<id> - what we hold
    GET   https://<host>/health              - no key, to check we are up

While a branch has no internet, keep both POSTs in an outbox on its
local server and send them when it returns. Late is fine.

Nothing about the message format, the reply codes, the key or the limits
changes between now and then. Please hold the host as configuration, not
a compiled-in constant: we will give you a staging host first and a
production host later.
```

PART B

Before you start

The checklist, the storage buckets, and what will be running

4 · What you need in hand

	What	Where from	Note
■	The two accounts from Part 0, with two-factor login on	Part 0	
■	**A domain name you control**	a UIU subdomain from UIU IT, or a registrar	**Required.** A certificate cannot be issued for a bare IP address
■	The three R2 buckets and their tokens	section 6 below	four values, each shown once
■	An SSH key pair	your laptop, step 1	password login will be turned off
■	CMED's code for each clinic	CMED	**needed at step 10**
■	The list of clinic laptops (clinic, room)	the clinics	**needed at step 11**
■	A password manager or the access sheet	AIMS LAB	**six secrets must be stored off the server** (step 6)

The domain is the only hard prerequisite. The gateway asks Let's Encrypt for a certificate the first time it starts, and that only works when the name already points at the droplet. This manual uses `aimscribe.uiu.ac.bd` as the example name; use whatever name you actually get.

5 · What will be running when you are done

Service	What it does	Exposed?
gateway (Caddy)	HTTPS, the certificate, security headers, a 2 MB body cap	**yes — 80 and 443**
api	every endpoint, on internal port 6000	no
postgres	both databases: `aims_recordings` and `aims_clinical`	no
pgbouncer	connection pooling	no
redis	the work queue	no
worker	transcription (needs the Azure settings; optional)	no
archive-worker	pulls the pieces of each recording, joins them, keeps the archive copy and uploads the encrypted cloud copy	no
backup	nightly dump of both databases, encrypted — **does not run in its image yet; step 12 replaces it**	no
monitor	disk, unconfirmed recordings, certificate, backups	no

Only the gateway is reachable from outside, on ports 80 and 443. Every other service talks on a private Docker network that has no route to the internet except through the gateway.

6 · Cloudflare R2: three buckets, three tokens

The stack needs three separate buckets, not one. The server refuses to make cloud copies if the first two settings name the same bucket (it logs a critical error and makes none), and the database backups must not share a token with anything else.

Bucket	What goes in it	Setting it fills
`aimscribe-segments`	The short pieces each recorder uploads while it records. The recorders upload straight to it, using a short-lived signed address from our API	`MINIO_BUCKET`
`aimscribe-archive`	One encrypted copy of each finished consultation — the permanent cloud archive (the 3 TB)	`AIMS_COPY_BUCKET`
`aimscribe-backups`	Each night's encrypted database backup, so a lost droplet does not take the backups with it. Kept 90 days; a few MB a night	`AIMS_OFFSITE_BUCKET`

	Do this in Cloudflare	Result
1	R2 Object Storage → **Create bucket** → name `aimscribe-segments` → Location: **Automatic**, with the hint **Asia-Pacific** if offered → Create	bucket 1
2	Same again for `aimscribe-archive` and for `aimscribe-backups`	buckets 2 and 3
3	Leave **Public access off** on all three. Never enable a public r2.dev address	private
4	R2 → **Manage R2 API Tokens** → Create API token → name `aimscribe-segments-rw` → permission **Object Read & Write** → **Apply to specific buckets only**: `aimscribe-segments` → Create	**Access Key ID** and **Secret Access Key** — shown once
5	Same again, named `aimscribe-archive-rw`, for `aimscribe-archive` only, and `aimscribe-backups-rw` for `aimscribe-backups` only	a second and a third pair — each shown once
6	Copy the **S3 endpoint** shown with the token. It looks like `https://<account-id>.r2.cloudflarestorage.com`	you will use it **without** the `https://`

If you see this instead	What it means, and what to do
The token page shows only "Account API tokens" and "User API tokens"	Either works. Pick **Object Read & Write** and restrict it to the one bucket.
The secret is gone after closing the page	It is shown once. Delete that token and create a new one.
A "Bucket lock" or retention option is offered on `aimscribe-archive`	Worth turning on for the archive bucket only: it stops anyone deleting the permanent copies. Never on the segments bucket, which the server empties as it works.

Keep the three pairs apart. The segments token can delete; the archive and backup tokens must never be given to anything except this server's settings. A leaked segments token then cannot reach the backups.

6b · The whole archive in Cloudflare

Every finished consultation ends up in `aimscribe-archive`, complete: the WAV and its clinical JSON, each encrypted on the droplet before it leaves. The droplet keeps its own copy only until the cloud copy is proven, plus two days. **The cloud is the archive**; the droplet is a short-term working copy. **Only a consultation CMED's server confirmed is copied**, so the bucket holds the dataset and nothing in it is ever erased for want of CMED (section 17d).

	How it works
In the bucket	`copies/<clinic>/<doctor>/<date>/<file name>.v1.wav.enc` and `.v1.json.enc`, one pair per consultation, in the same folders as the archive. A prescription that arrives later adds `.v2.json.enc`; nothing is overwritten
Who can read it	**Nobody without** `AIMS_COPY_KEY` — not Cloudflare, not anyone holding the token. AES-GCM, and each copy is decrypted and hashed again before it is uploaded
When a copy counts	The server reads the stored object back (size and fingerprint) before it lets the uploaded pieces or the local WAV go

	How it works
The droplet's copy	`AIMS_ARCHIVE_LOCAL_RETENTION_DAYS=0`: a WAV leaves the droplet within 10 minutes of its copy being proven, never before. The manifest, JSON and catalogue stay, and say where the copy is
Getting one back	Download the `.enc` files (the Cloudflare dashboard, or `rclone`), then on any PC with Python: `python backend/archive_worker/restore.py restore <file or folder> <out folder>` with `AIMS_COPY_KEY` set. `restore.py check` compares the result with the catalogue's SHA-256
Protection	Bucket lock (a retention rule) on `aimscribe-archive` only, so a copy cannot be deleted early; its token reaches that bucket and nothing else
Cost	$15 per TB a month; downloads are free. 3 TB is $45

Practise getting one back
In step 12b's restore drill, also download one `.wav.enc` from `aimscribe-archive`, restore it on your laptop and play it. An archive nobody has restored from is a hope, not an archive.

7 · What the 16 GB droplet needs that the default does not

The stack's defaults were written for a 64 GB machine at UIU. On the 16 GB droplet four settings must change, or the database will not stay up. The profile they point at ships with the code; step 6e sets them:

Setting	Default	On the droplet	Why
`AIMS_PG_CONF`	`postgresql.conf`	`postgresql.16gb.conf`	**The default reserves 16 GB for the database cache alone** — the whole machine. The 16 GB profile is in `deploy/uiu/postgres`, tested under a 6 GB limit
`AIMS_PG_MEMORY`	8g	**6g**	leaves room for the API and the workers
`AIMS_ARCHIVE_MEMORY`	1g	**2g**	joining a long consultation in memory, without being killed
`AIMS_WORK_TMPFS`	6g	**3g**	working space in memory for joining one consultation

PART C

The deployment

Twelve steps, three to four hours, done once

Step 1 · Make your SSH key, then create the droplet

An SSH key is how you log in to the server without a password. You make it once, on your own laptop. Windows 10 and 11 already include the tool.

1a · On your laptop, in Windows PowerShell

```
PS> ssh-keygen -t ed25519 -C "your-team-email@example.org"
#   press Enter to accept the file name
#   type a passphrase (twice) - it protects the key if the laptop is stolen

PS> Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | Set-Clipboard
#   the PUBLIC key is now on the clipboard
```

You should see: two files in `C:\Users\<you>\.ssh\`: `id_ed25519` (private — never share it, never upload it) and `id_ed25519.pub` (public — this is what DigitalOcean gets). The public key is one line starting `ssh-ed25519 AAAA`.

If you see this instead	What it means, and what to do
'ssh-keygen' is not recognized	Settings → System → Optional features → Add a feature → **OpenSSH Client**. Then open a new PowerShell window.
"id_ed25519 already exists. Overwrite?"	Answer **n**. You already have a key; use the existing `.pub` file.

1b · In DigitalOcean: Create → Droplets

Field	Choose
Region	**Bangalore · BLR1** — the lowest latency to Dhaka
Image	**Ubuntu 24.04 (LTS) x64**
Droplet type	**Basic** → **Regular (SSD)**
Size	**8 vCPU / 16 GB / 320 GB SSD —** `s-8vcpu-16gb` **— about $96 a month**
Authentication	**SSH Key** → **New SSH Key** → paste the clipboard → name it after yourself. **Not a password**
Backups	**Enable — weekly** (about 20% extra, ~$19 a month)
Monitoring	**Enable** — free; needed for the email alerts in step 12
Hostname	`aimscribe-prod-blr1`
Project	AIMScribe

You should see: after about a minute the droplet appears with a green dot and a **public IPv4 address**. Write the address down; it is `<droplet-ip>` below.

Optional: a reserved IP. Networking → Reserved IPs → assign one to the droplet, and use that address from now on. It is free while attached, and if the droplet is ever rebuilt the address — and so the DNS record — stays the same.

Step 2 · Point the name at the droplet

At whoever controls the domain (UIU IT for a `uiu.ac.bd` name), create one record:

```
Type  Name                  Value              TTL
A     aimscribe.uiu.ac.bd   <droplet-ip>       300
```

Then wait, and check from your laptop:

```
PS> nslookup aimscribe.uiu.ac.bd 1.1.1.1
```

You should see: `Address: <droplet-ip>` in the answer. Usually within 5 minutes; occasionally up to an hour.

If you see this instead	What it means, and what to do
`Non-existent domain` **or no address**	The record is not live yet, or was typed under the wrong name. Wait 10 minutes and try again; then check the record itself.
A different address	An old record exists. Delete it so only the new A record remains.

Do not start the stack (step 8) until this answers correctly. The gateway asks for a certificate on first start; if the name does not point here yet the request fails and Let's Encrypt makes you wait before trying again.

Step 3 · The firewall

DigitalOcean → Networking → **Firewalls** → Create Firewall, name `aimscribe-prod`, with exactly these rules, then apply it to the droplet:

Direction	Protocol	Port	Source / destination	Why
Inbound	TCP	**22**	**your own IP only**	SSH, administration
Inbound	TCP	**80**	All IPv4, All IPv6	certificate issue and renewal
Inbound	TCP	**443**	All IPv4, All IPv6	**the API**
Inbound	UDP	**443**	All IPv4, All IPv6	HTTP/3
Outbound	all	all	all	R2 uploads, updates, certificates

Nothing else. No database port, no Redis port, no API port. Use this DigitalOcean firewall rather than `ufw` on the droplet: Docker opens its ports around `ufw`, so `ufw` alone would give a false sense of safety.

If you see this instead	What it means, and what to do
You do not know your own IP	Search "what is my IP" from the laptop you will administer from, and use that address with `/32`.
Your IP changes (home broadband)	Add each address you use, or use the DigitalOcean web console (droplet → Access) when locked out.

Step 4 · First login, a working user, and Docker

From your laptop, log in as `root` the first time only:

```
PS> ssh root@<droplet-ip>
#   first time: "Are you sure you want to continue connecting?" -> yes
#   then your key passphrase
```

You should see: `root@aimscribe-prod-blr1:~#` — you are on the server.

Now, on the droplet, create the user you will use from now on, and lock the door behind it:

```
# a working user with sudo, and your key
adduser --disabled-password --gecos "" aims
usermod -aG sudo aims
rsync --archive --chown=aims:aims ~/.ssh /home/aims
passwd aims            # set a password - sudo will ask for it

# no root login, no password logins
sed -i "s/^#*PermitRootLogin.*/PermitRootLogin no/" /etc/ssh/sshd_config
sed -i "s/^#*PasswordAuthentication.*/PasswordAuthentication no/" /etc/ssh/sshd_config
grep -rl "PasswordAuthentication yes" /etc/ssh/sshd_config.d/ \
  | xargs -r sed -i "s/PasswordAuthentication yes/PasswordAuthentication no/"
sshd -t && systemctl restart ssh
sshd -T | grep -E "^(permitrootlogin|passwordauthentication)"
```

You should see: `permitrootlogin no` and `passwordauthentication no`.

Keep this root window open until the next login works
Open a **second** PowerShell window and run `ssh aims@<droplet-ip>`. Only when that succeeds, close the root window. If it fails, the root window is still open to fix it — and the DigitalOcean web console (droplet → Access → Launch Droplet Console) always works as a last resort.

As `aims`, the updates, the clock, a swap file and Docker:

```
ssh aims@<droplet-ip>

sudo apt update && sudo apt upgrade -y
sudo apt install -y unattended-upgrades fail2ban git curl jq python3
sudo dpkg-reconfigure -plow unattended-upgrades      # answer Yes

# the clock - recordings are filed by the clinic's local date
sudo timedatectl set-timezone Asia/Dhaka
sudo timedatectl set-ntp true

# 4 GB of swap: a cushion, so a spike never kills the database
sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo "/swapfile none swap sw 0 0" | sudo tee -a /etc/fstab

# Docker, with the compose plugin
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker aims
sudo systemctl enable --now docker
exit                                  # log out and back in, so the group applies
ssh aims@<droplet-ip>
docker compose version && timedatectl | grep "Time zone" && free -h
```

You should see: `Docker Compose version v2...`, `Time zone: Asia/Dhaka`, and `Swap: 4.0Gi`.

If you see this instead	What it means, and what to do
`permission denied ... docker.sock`	You did not log out and back in after `usermod`. Run `exit`, then log in again.
`Could not get lock /var/lib/dpkg/lock`	Ubuntu is installing its own updates in the background on first boot. Wait five minutes and run the command again.
A purple screen asks which services to restart	Press Enter to accept the default.

The timezone line is not cosmetic. A recording is filed under the clinic's local date. A server on UTC files the evening's consultations under tomorrow.

Step 5 · Get the code

```
cd ~
git clone https://github.com/SakibZaman021/AIMScribe_v3.git
cd AIMScribe_v3/deploy/uiu
ls
```

You should see: `Caddyfile backup docker-compose.yml monitor pgbouncer postgres ...` and, with `ls -a`, `.env.example`.

The folder is named `uiu` for historical reasons. It is the **server** stack and the right one for this droplet. `deploy/local` is the development stack; never use it here.

Step 6 · The settings file

All settings live in one file, `deploy/uiu/.env`. It holds every secret on the machine, is readable only by you, and is **never committed** — the repository is public. Work through 6a to 6g in one SSH session, in the `deploy/uiu` folder.

6a · Start from the example, and generate the secrets on the server

```
cd ~/AIMScribe_v3/deploy/uiu
cp .env.example .env && chmod 600 .env

# setv NAME VALUE - replaces the line NAME=... in .env, or adds it
setv() { sed -i "s|^$1=.*|$1=$2|" .env; grep -q "^$1=" .env || echo "$1=$2" >> .env; }

# passwords and service keys: random, 64 hex characters each, from the
# system's secure random source. Nothing is ever left at a default.
for v in POSTGRES_SUPERUSER_PASSWORD AIMS_API_DB_PASSWORD \
         AIMS_CLINICAL_PASSWORD AIMS_MONITOR_PASSWORD REDIS_PASSWORD \
         AIMS_ADMIN_KEY AIMS_WORKER_KEY AIMSCRIBE_WEBHOOK_SECRET; do
  setv $v "$(openssl rand -hex 32)"
done

# the two encryption keys: 32 random bytes each, base64
setv AIMS_BACKUP_KEY "$(openssl rand -base64 32)"
setv AIMS_COPY_KEY   "$(openssl rand -base64 32)"
```

Keep this SSH window open until 6g: `setv` exists only in this session. If the window closes, paste the `setv()` line again before continuing. Hex is used for the passwords because one of them is placed inside a database address, where `/` or `+` would break it.

6b · Who this server is

```
setv AIMS_PUBLIC_HOST aimscribe.uiu.ac.bd             # the name from step 2
setv AIMS_ACME_EMAIL  your-team-email@example.org
setv TZ               Asia/Dhaka
setv AIMS_DB_PATH      /srv/aimscribe/db
setv AIMS_ARCHIVE_PATH /srv/aimscribe/archive
setv AIMS_BACKUP_PATH  /srv/aimscribe/backup

# the exact address CMED's dashboard is served from (no wildcards);
# ask CMED - a placeholder is harmless, it only governs browser access
setv AIMS_ALLOWED_ORIGINS https://<cmed-dashboard-host>

# OPTIONAL: the lab's fixed public address, if it has one. The dashboard
# and /api/v2/admin then also open from there. Left empty they open ONLY
# through the SSH tunnel (section 13) - the safer default.
# setv AIMS_ADMIN_ALLOW <lab public IP>/32
```

The admin routes have two locks. The admin key, and where the request comes from: from the internet `/api/v2/admin` and the dashboard answer `403` unless the address is in `AIMS_ADMIN_ALLOW`. On the droplet itself they are on `http://127.0.0.1:6001`, which is how step 10 and section 10 reach them.

6c · The three R2 buckets from Part B, section 6

```
# the endpoint is the HOST NAME ONLY - no https:// in front
R2=<account-id>.r2.cloudflarestorage.com

# the pieces, uploaded by the recorders while they record
setv MINIO_ENDPOINT   $R2
setv MINIO_BUCKET     aimscribe-segments
setv MINIO_SECURE     true
setv MINIO_REGION     auto
setv MINIO_ACCESS_KEY <segments token Access Key ID>
setv MINIO_SECRET_KEY <segments token Secret Access Key>

# the permanent encrypted copy of each consultation
setv AIMS_COPY_ENDPOINT   $R2
setv AIMS_COPY_BUCKET     aimscribe-archive
setv AIMS_COPY_REGION     auto
setv AIMS_COPY_SECURE     true
setv AIMS_COPY_ACCESS_KEY <archive token Access Key ID>
setv AIMS_COPY_SECRET_KEY <archive token Secret Access Key>

# each night's encrypted database backup, off the droplet (this one WITH https://)
setv AIMS_OFFSITE_ENDPOINT   https://$R2
setv AIMS_OFFSITE_BUCKET     aimscribe-backups
setv AIMS_OFFSITE_ACCESS_KEY <backups token Access Key ID>
setv AIMS_OFFSITE_SECRET_KEY <backups token Secret Access Key>
```

`.env.example` **suggests Amazon for the copy store (**`s3.ap-south-1.amazonaws.com`**).** The lines above replace that with R2, which is the policy for this deployment.

6d · The server's two signing keys

The server signs two things with private keys: each **recording grant**, and each **purge receipt** that lets a clinic PC delete audio it has safely uploaded. Every recorder checks those signatures with the matching **public** keys, which the installer copies to `C:\ProgramData\AIMScribe\keys`. **The private and public halves must be a pair**, or no recording can be authorised and no audio is ever deleted from a PC.

Make a new pair on the droplet, put the private halves in `.env`, and keep the public halves for the installer:

```
mkdir -p ~/keys && cd ~/keys && umask 077
openssl genpkey -algorithm ed25519 -out grant.pem
openssl genpkey -algorithm ed25519 -out receipt.pem
openssl pkey -in grant.pem   -pubout -out aimslab_grant_pub.pem
openssl pkey -in receipt.pem -pubout -out aimslab_receipt_pub.pem

cd ~/AIMScribe_v3/deploy/uiu
# .env holds a key on one line, with its line breaks written as \n
for k in GRANT RECEIPT; do
  f=~/keys/$(echo $k | tr A-Z a-z).pem
  sed -i "/^AIMS_${k}_PRIVATE_KEY=/d" .env
  printf "%s\n" "AIMS_${k}_PRIVATE_KEY=$(awk 'BEGIN{ORS="\\n"}1' $f)" >> .env
done

# prove each private key in .env matches its public file
for k in grant receipt; do
  K=$(echo $k | tr a-z A-Z)
  grep "^AIMS_${K}_PRIVATE_KEY=" .env | cut -d= -f2- | xargs -0 printf "%b" \
    | openssl pkey -pubout | diff -q - ~/keys/aimslab_${k}_pub.pem >/dev/null \
    && echo "$k: MATCH" || echo "$k: DIFFERENT"
done
```

You should see: `grant: MATCH` and `receipt: MATCH`.

Then copy the two **public** files to your laptop, into the recorder source, so the installer you build in step 11 carries them:

```
PS> cd <your AIMScribe_v3 folder>\recorder\keys
PS> scp aims@<droplet-ip>:keys/aimslab_grant_pub.pem .
PS> scp aims@<droplet-ip>:keys/aimslab_receipt_pub.pem .
```

Store `grant.pem` and `receipt.pem` in the password manager (6g), then delete them from the droplet with `shred -u ~/keys/grant.pem ~/keys/receipt.pem`. The copies in `.env` are the working ones.

Why new keys, and not the ones already in the repository
On 15 October 2026 the public keys in `recorder/keys` did **not** match the private keys of the AIMS LAB development server. Nobody can say for certain which server holds their private halves. Starting with a fresh pair removes the doubt, and costs nothing: every clinic PC is re-run through the installer in step 11 anyway, which installs the new public keys.

6e · Sizing for the 16 GB droplet

The database profile for this droplet is part of the code: `deploy/uiu/postgres/postgresql.16gb.conf`. Its header shows how the 16 GB is shared between the services. Point the settings at it:

```
cd ~/AIMScribe_v3/deploy/uiu
grep -E "^(shared_buffers|effective_cache_size|work_mem)" postgres/postgresql.16gb.conf

setv AIMS_PG_CONF        ./postgres/postgresql.16gb.conf
setv AIMS_PG_MEMORY      6g
setv AIMS_ARCHIVE_MEMORY 2g
setv AIMS_WORK_TMPFS     3g

# how long an archived WAV stays on this disk once its cloud copy is proven
setv AIMS_ARCHIVE_LOCAL_RETENTION_DAYS 0       # no audio kept on the droplet
setv AIMS_BACKUP_KEEP_DAYS 7                  # a week here; 90 days in Cloudflare
```

You should see: `shared_buffers = 4GB`, `effective_cache_size = 10GB`, `work_mem = 16MB`.

The disk floors need nothing. Left empty, each of the monitor's floors is capped at 15% of its disk — 48 GB here — so a floor written for the UIU array no longer alarms for ever on a 320 GB disk.

6f · Alerts, and the optional parts

```
# where the monitor posts its alerts (a Slack/Discord/Google Chat webhook URL).
# Left empty, alerts go only to the log.
setv AIMS_ALERT_WEBHOOK https://<your chat webhook address>

# transcription: only if this deployment transcribes. Otherwise leave empty.
#   AZURE_TRANSCRIBE_ENDPOINT, AZURE_TRANSCRIBE_API_KEY,
#   AZURE_NER_ENDPOINT, AZURE_NER_API_KEY
```

6g · Check, then put six secrets somewhere safe

```
for v in POSTGRES_SUPERUSER_PASSWORD AIMS_API_DB_PASSWORD AIMS_CLINICAL_PASSWORD \
  AIMS_MONITOR_PASSWORD REDIS_PASSWORD AIMS_WORKER_KEY AIMS_ADMIN_KEY AIMS_PUBLIC_HOST \
  AIMS_ACME_EMAIL AIMS_DB_PATH AIMS_ARCHIVE_PATH AIMS_BACKUP_PATH AIMS_BACKUP_KEY \
  AIMS_COPY_KEY AIMS_GRANT_PRIVATE_KEY AIMS_RECEIPT_PRIVATE_KEY MINIO_ENDPOINT \
  MINIO_ACCESS_KEY MINIO_SECRET_KEY AIMS_COPY_BUCKET AIMS_COPY_ACCESS_KEY \
  AIMS_COPY_SECRET_KEY AIMS_PG_CONF; do
  grep -q "^$v=." .env && echo "ok       $v" || echo "MISSING  $v"
done
grep -c "^MINIO_BUCKET=aimscribe-segments$" .env     # must print 1
docker compose config --quiet && echo "compose settings: OK"
```

You should see: every line `ok`, then `1`, then `compose settings: OK`. No secret is printed by these checks.

If you see this instead	What it means, and what to do
`MISSING <name>`	Set it with `setv` as above, then run the check again.
`required variable ... is missing a value`	One of the eighteen the stack refuses to start without is empty - now including `AIMS_BACKUP_KEY`, both signing keys and the segment store's. The message names it.
`setv: command not found`	The session was closed. Paste the `setv()` line from 6a again.

Six secrets that must also exist OFF this machine	If lost
`AIMS_COPY_KEY`	**Every cloud copy in R2 is permanently unreadable**
`AIMS_BACKUP_KEY`	**Every database backup is unreadable**
`grant.pem` / `AIMS_GRANT_PRIVATE_KEY`	every clinic PC must be re-installed with new public keys

Six secrets that must also exist OFF this machine	If lost
`receipt.pem` / `AIMS_RECEIPT_PRIVATE_KEY`	same
`AIMS_ADMIN_KEY`	no administration until `.env` is read on the server
`POSTGRES_SUPERUSER_PASSWORD`	restores and repairs become much harder

To read a value to copy it into the password manager, without leaving it in the shell history: `grep ^AIMS_COPY_KEY= .env`. Do this on your own screen, never in a screen share.

Step 7 · The folders, on an encrypted volume

First, the data volume. In the DigitalOcean panel: Volumes → Create Volume, 100 GB, same region as the droplet, attach it to the droplet, **Manually format & mount**. DigitalOcean encrypts volume data at rest. The databases, the archive and the backups then live on it, not on the droplet's own disk:

```
ls /dev/disk/by-id/ | grep DO_Volume          # the new volume
V=/dev/disk/by-id/scsi-0DO_Volume_<name>
sudo mkfs.ext4 -L aimsdata "$V"               # once, on a NEW volume only
sudo mkdir -p /srv/aimscribe
echo "$V /srv/aimscribe ext4 defaults,nofail,discard,noatime 0 2" \
  | sudo tee -a /etc/fstab
sudo mount -a && df -h /srv/aimscribe
```

```
sudo mkdir -p /srv/aimscribe/{db,archive,backup}
# the archive worker runs as user 10001 inside its container, and must
# be able to write the archive; the database and backup containers
# set their own ownership
sudo chown -R 10001:10001 /srv/aimscribe/archive
# the nightly backup in step 12 runs as you
sudo chown aims:aims /srv/aimscribe/backup
df -h /srv
```

You should see: the volume mounted on `/srv/aimscribe` with about `98G` available.

Do not `chown` **the archive folder to** `aims`**.** The archive worker would then fail to write any recording, with `Permission denied` in its log.

Step 8 · Start it

```
cd ~/AIMScribe_v3/deploy/uiu
docker compose up -d --build
#   the first time builds four images: 5-10 minutes

docker compose ps
```

You should see: nine services. `aims-postgres`, `aims-pgbouncer`, `aims-redis` and `aims-api` say `(healthy)`; the rest say `Up`. The first start creates both databases and applies the whole schema by itself.

```
docker compose logs --tail 60 api
docker compose logs -f gateway        # Ctrl+C to stop watching
```

You should see: in the gateway log, `certificate obtained successfully` for your host, within a minute or two.

If you see this instead	What it means, and what to do
`aims-postgres` **restarting, or its log says** `could not map anonymous shared memory`	The 16 GB profile is not in use. Check `AIMS_PG_CONF` (6e), then `docker compose up -d`.
`aims-postgres` **log:** `could not open configuration file`	The file named in `AIMS_PG_CONF` does not exist. Check the spelling in 6e, and that `git pull` brought `postgres/postgresql.16gb.conf`.
Gateway log: `NXDOMAIN`, `timeout` **or** `rateLimited`	DNS is not pointing here yet (step 2), or port 80 is closed (step 3). Fix it, then `docker compose restart gateway`. After a rate limit, wait an hour.

If you see this instead	What it means, and what to do
API log: `AIMS_COPY_BUCKET is the same bucket as the pieces`	Both settings name one bucket. Set them as in 6c, then `docker compose up -d`.
API log: `AIMS_RECEIPT_PRIVATE_KEY is not set`	6d was not completed.
Archive-worker log: `Permission denied` **on** `/archive`	Step 7's chown was skipped or used the wrong user.
A service is `Exited`	`docker compose logs --tail 80 <service>` shows why. Fix the setting, then `docker compose up -d`.

Step 9 · Prove it answers — from your laptop, not the server

```
PS> curl.exe -s https://aimscribe.uiu.ac.bd/health

# and that what must stay closed is closed: 404, 404, 404, 403
PS> foreach ($p in "/api/v1/transcript/x", "/docs", "/api/v2/archive/pending",
                   "/api/v2/admin/alerts") {
      curl.exe -s -o NUL -w "%{http_code}  $p`n" https://aimscribe.uiu.ac.bd$p }
```

You should see: first `{"status":"healthy","system":"AIMScribe v3 ..."}` - from outside, /health shows only those two (the detail is for the server's own checks), then `404` for the old v1 API, the API map and the archive worker's route, and `403` for admin - unless your laptop's address is in `AIMS_ADMIN_ALLOW`. Any `200` there is a door left open: stop and check the Caddyfile is the one from this release.

You should see: `"status":"healthy"`, `"database":"connected"`, `"redis":"connected"`, `"minio":"connected"`, `"copy_bucket":"connected"`, and `"system"` starting with `AIMScribe v3`. The two storage answers are real: the server asks each R2 bucket, with its own keys, at most every 30 seconds.

If you see this instead	What it means, and what to do
Could not resolve host	Step 2 is not finished.
A certificate or TLS error	The gateway has not got its certificate yet. Watch `docker compose logs -f gateway`.
`"status":"degraded"`	Read which part is not `connected`. The database or Redis: `docker compose ps`. `"minio"` or `"copy_bucket"` saying `refused`: that bucket's keys are wrong (6c); `bucket missing`: the bucket name is wrong; `unreachable`: the endpoint is wrong.
Connection refused or timed out	Port 443 is closed in the firewall (step 3), or the gateway is not running.

Run it from somewhere else, not from the droplet. Testing from the server proves the program is running; it does not prove that DNS, the firewall and the certificate work — which is the point of this step.

Step 10 · Register the clinics, the doctors and CMED's key

These are admin API calls, authenticated with `X-Admin-Key`. Run them on the droplet; the key is read from `.env` into a variable and never typed. They go to the gateway's loopback-only door, `127.0.0.1:6001`: from the internet the admin routes answer `403`.

```
cd ~/AIMScribe_v3/deploy/uiu
A=$(grep ^AIMS_ADMIN_KEY= .env | cut -d= -f2)
H=http://127.0.0.1:6001/api/v2          # the lab door, this machine only

# one clinic. cmed_hospital_id is CMED's code for it - the mapping that
# lets CMED's visits be matched to this clinic's recordings
curl -s -X POST $H/admin/hospital -H "X-Admin-Key: $A" \
  -H "Content-Type: application/json" \
  -d '{ "hospital_id": "AALO_DHOLPUR",
        "name": "Aalo Clinic, Dholpur",
        "timezone": "Asia/Dhaka",
        "cmed_hospital_id": "<CMED code for this clinic>" }'

# repeat for AALO_KARAIL, AALO_MIRPUR, AALO_SHYAMPUR,
```

```
#            AALO_NARAYANGANJ, AALO_ERSHADNAGAR, AMADER_SUSASTHO

# a doctor (optional - CMED names the doctor with every consultation;
# this only puts a readable name against the id in reports)
curl -s -X POST $H/admin/doctor -H "X-Admin-Key: $A" \
  -H "Content-Type: application/json" \
  -d '{ "doctor_id": "DR0042", "hospital_id": "AALO_DHOLPUR",
        "full_name": "Dr Rahman", "active": true }'

# CMED, one client per hospital. CMED makes each key pair and sends only
# the PUBLIC key file; the private key never leaves CMED's server.
export AIMS_ADMIN_KEY="$A"
python3 ../../tools/cmed_fhir_client.py register --server http://127.0.0.1:6001 \
  --client-id cmed-aalo-dholpur --hospital "<CMED code for this clinic>" \
  --jwk cmed-aalo-dholpur.public.json --cidr "<CMED server address>/32"
# repeat for each clinic; send CMED each client_id and kid
```

You should see: each clinic: `{"status":"ok","hospital_id":"AALO_DHOLPUR","cmed_hospital_id":"..."}`. The key call: `{"client_id":"cmed-aalo-dholpur","kid":"...",...}`.

If you see this instead	What it means, and what to do
`401 invalid admin key`	The A= line was run in another folder. `cd` into `deploy/uiu` and run it again.
`422` **with** `field required`	The JSON body is missing a field, or a quote was lost in copying. Retype the body.
`400 timezone ... is not a known zone name`	Use `Asia/Dhaka` exactly.

Nothing secret changes hands here. CMED sends public keys; we send back client ids and key ids. To rotate a hospital's key, register the new public key with `POST $H/admin/cmed-client/<id>/key`, let CMED switch, then revoke the old one. To cut a hospital off at once: `POST $H/admin/cmed-client/<id>/revoke`. The old static key stays off (`AIMS_CMED_LEGACY_KEYS` empty).

`cmed_hospital_id` **is the blocking prerequisite**
Without it, a visit CMED describes cannot be tied to the clinic it happened in: the record is stored, **CMED receives a normal 202**, an `unmapped_clinic` alert is raised, and the recording is kept but never gets its clinical record. **Get CMED's seven codes before this step.** If they are late, register the clinics anyway and repeat the same call with the code later; it updates, it does not duplicate.

Step 11 · Install the clinic PCs against the new address

A recorder is enrolled to **one clinic and one server**. **From recorder 3.7 no token is needed:** run the installer with the server address, and the PC joins by itself at its first CMED consultation, at the clinic CMED names (section 17c) — it needs the internet at that moment, once. The tokens below remain for PCs IT wants enrolled ahead of time. **Do one PC first, record a full test consultation, and only then do the rest.**

11a · Mint the tokens (on the droplet)

The fleet is described in a CSV, `backend/scripts/laptops.csv`: **one row per laptop, not per doctor**. **Add rows; never delete rows** — a trimmed file reads as clinics that no longer exist.

```
#   hospital_id,hospital_name,room,doctor_id,doctor_name
#   AALO_DHOLPUR,"Aalo Clinic, Dholpur",Room 3,,
#   AALO_DHOLPUR,"Aalo Clinic, Dholpur",Room 4,,
nano ~/AIMScribe_v3/backend/scripts/laptops.csv      # Ctrl+O, Enter, Ctrl+X

cd ~/AIMScribe_v3/backend
AIMS_BACKEND_URL=https://aimscribe.uiu.ac.bd \
AIMS_ADMIN_URL=http://127.0.0.1:6001 \
AIMS_ADMIN_KEY=$(grep ^AIMS_ADMIN_KEY= ../deploy/uiu/.env | cut -d= -f2) \
  python3 scripts/mint_enrolment_tokens.py scripts/laptops.csv
```

You should see: `backend https://aimscribe.uiu.ac.bd`, one `hospital ... ready` per clinic, and a folder `scripts/enrolment_<date>_<time>/` with one instruction sheet per PC. Unused tokens expire after 14

days.

If you see this instead	What it means, and what to do
`AIMS_BACKEND_URL is not set`	Run it again exactly as shown, with the `AIMS_BACKEND_URL=` line.
`AIMS_ADMIN_KEY is not set`	The path to `.env` was wrong. Run it from `~/AIMScribe_v3/backend`.

Copy the instruction sheets to your laptop with `scp -r`, then **delete the folder from the droplet**: the tokens are credentials until they are used.

11b · Prove the keys belong together, then build the installer

REQUIRED before building the installer and before enrolling any PC
On 15 October 2026 the public keys in the repository's `recorder/keys` did not belong to the development server's private keys. An installer built from them makes **every recording fail its grant check**, and no PC ever frees its disk. Three fingerprints must be the same: the server's, the installer's, and the first PC's. No private key leaves the droplet to check this.

1. On the droplet: the server's private keys against the public halves made in 6d.

```
cd ~/AIMScribe_v3/deploy/uiu
docker compose run --rm --no-deps --user root \
  -v ~/keys:/keys:ro -v ~/AIMScribe_v3/tools:/tools:ro \
  api python /tools/check_key_pair.py --from-env --keys /keys
```

You should see: `grant MATCH <16 characters>` and the same for `receipt`. Write the two fingerprints down.

2. On your laptop, with the two files from 6d in `recorder\keys`:

```
PS> cd <your AIMScribe_v3 folder>
PS> python tools\check_key_pair.py --show --keys recorder\keys
```

You should see: the same two fingerprints as on the droplet. **If either differs, stop**: copy the files from 6d again. The installer copies these keys to every PC it runs on.

Build the signed installer once, with this server's address in it, so nobody types it on twenty PCs. Inno Setup 6 and the Windows SDK's signtool must be on your laptop, and the code-signing certificate set up as in `recorder/INSTALL_FOR_IT.md`. An unsigned installer is what makes clinic laptops block it; never ask a clinic to switch Windows security off. Ask CMED for the exact address their doctor page is opened from:

```
C:\> cd <your AIMScribe_v3 folder>\recorder
C:\> set AIMS_SIGN_PROVIDER=thumbprint     (or azure / signpath)
C:\> set AIMS_SIGN_THUMBPRINT=<certificate thumbprint>
C:\> set AIMS_BACKEND=https://aimscribe.uiu.ac.bd
C:\> set AIMS_ORIGIN=https://<cmed-doctor-page-host>
C:\> BUILD.bat        # tests, build, sign every file, signed installer
```

You should see: `dist\AIMScribeSetup.exe`, version 3.5.0, signed by AIMS LAB, with `AIMScribeSetup.exe.sha256` beside it. BUILD.bat stops if any shipped file is unsigned. That one file goes on every clinic PC; an upgrade keeps each PC's own settings and key. IT can install it silently: `AIMScribeSetup.exe /VERYSILENT /SUPPRESSMSGBOXES /NORESTART /BACKEND=https://aimscribe.uiu.ac.bd /ORIGIN=https://<cmed page> /TOKEN=<this PC's token>`. Then run `AIMScribe - check this PC` from the Start menu: every row PASS.

3. On the first clinic PC, after 11c: `python tools\check_key_pair.py --show --keys C:\ProgramData\AIMScribe\keys` (from a copy of the tools folder) shows the same two fingerprints.

11c · On each clinic PC

	Do this	Note
1	**Only for a PC already enrolled to another AIMS LAB server:** wait until it has nothing left to upload (the folder `C:\ProgramData\AIMScribe\spool` has no session folders). Then delete `C:\ProgramData\AIMScribe\state\device.json`	**Without this the PC stays enrolled to the old server** and ignores the new token. A brand-new PC skips this row
2	Run `AIMScribeSetup.exe` as administrator	
3	**Backend URL:** already filled in — check it reads `https://aimscribe.uiu.ac.bd`	built in by 11b
4	**CMED web address:** already filled in — the address CMED's doctor page is opened from	built in by 11b
5	**Enrolment token:** that PC's token from its sheet	one PC, one token
6	Finish. The agent starts and enrols itself on first start	needs the internet once

You should see: on the droplet, `docker compose logs api | grep -i enrol` shows the PC enrolling, and from then on its heartbeat every 30 seconds.

If you see this instead	What it means, and what to do
The installer says "leave empty — already enrolled"	Row 1 was skipped. Cancel, delete `device.json`, run it again.
The PC refuses to record: "not registered with AIMS LAB"	Enrolment failed. Usually an expired or already-used token: mint a new one for that row.
Recording starts but is never authorised	The PC's public keys do not match `.env`. Run the three checks in 11b, rebuild the installer with the files from 6d, and run it again on that PC.

Tell CMED the new host for their server's setting (Part A, section 3). That is the only change on their side.

Step 12 · Backups, the restore drill, and email alerts

The stack's `backup` service makes an encrypted dump of both databases every night at `AIMS_BACKUP_AT` (02:30 Dhaka time) into `/srv/aimscribe/backup`, and keeps 30 days. It runs in its own small image, `deploy/uiu/backup/Dockerfile`: the plain `postgres:16-alpine` it used before could neither read its schedule nor encrypt, and never made a backup. Make one now, to prove it, rather than waiting for tonight.

12a · The first backup, now

```
cd ~/AIMScribe_v3/deploy/uiu
docker compose run --rm backup --now
ls -lh /srv/aimscribe/backup
docker compose logs --tail 3 backup        # the nightly service's own plan
```

You should see: `aims_recordings -> aims_recordings_<date>_<time>.sql.gz.enc` and the same for `aims_clinical`, then `done; 2 backup file(s) held`. The service's log ends `next backup in <h>h <m>m` — a real wait, never a negative one.

If you see this instead	What it means, and what to do
`service "postgres" is not running`	Start the stack first (step 8).
`Permission denied` **writing** `/backups`	Step 7's last chown was skipped.
`is only ... bytes; keeping it for inspection`	The dump failed or the database is empty. Read the first error above it.
`AIMS_BACKUP_KEY is set but this image has no openssl`	The service is still on the plain image. `docker compose up -d --build backup`.

12b · The restore drill

Restore the newest backup into a **scratch** database, count what is in it, and drop it. The script refuses to write into either live database.

```
cd ~/AIMScribe_v3/deploy/uiu
F=$(ls -t /srv/aimscribe/backup/aims_recordings_*.enc | head -1); echo $F
docker compose run --rm backup restore /backups/$(basename $F) aims_restore_test

docker compose exec -T postgres psql -U aims_admin -d postgres \
  -c "DROP DATABASE aims_restore_test"
```

You should see: `Decompressed ...: <n> lines`, `CREATE DATABASE`, then a table of row counts with `sessions` matching the dashboard, then `DROP DATABASE`.

If you see this instead	What it means, and what to do
`bad decrypt`	The key in `.env` is not the one the file was made with. This is exactly why the key is stored off the machine.
`Restore failed; last lines:`	Read them. Do not ignore it: fix it today.
`Refusing to restore over the live database`	As intended. Restore into a new name, check it, then swap it in by hand.

A backup nobody has restored is not a backup. Do this drill today, then once a month.

12b2 · The copy off the droplet

Each night, after the dump, the same job copies every encrypted backup to `aimscribe-backups`, checks the size of each copy, and keeps them 90 days. A night that cannot reach R2 is caught up the next night. Only encrypted backups ever leave the droplet. Check it now:

```
docker compose run --rm backup list-offsite
# the drill, from the off-site copy - as if the droplet were gone
docker compose run --rm backup fetch latest aims_recordings
docker compose run --rm backup restore /backups/<the file it printed> aims_restore_test
```

You should see: the two files from 12a, each with its size, and the log line `off-site: aims_recordings_... (size checked)`. The monitor alerts if no copy has reached the bucket for 36 hours.

If you see this instead	What it means, and what to do
`no off-site copy is configured`	The four `AIMS_OFFSITE_` lines from 6c are missing. Add them, then `docker compose up -d backup`.
`off-site copy refused - the backups are not encrypted`	`AIMS_BACKUP_KEY` is empty. Backups hold patients' names and never leave unencrypted.
`AccessDenied` **or** `403`	The backups token is not limited to, or not allowed to write, `aimscribe-backups`.

12c · Email alerts from DigitalOcean

	Do this in DigitalOcean	Emails the owner when
1	Monitoring → **Create alert policy**: Disk utilisation above **80%** for 5 min, droplet `aimscribe-prod-blr1`	the disk is filling
2	Same: Memory above **90%** for 10 min	memory is short
3	Same: CPU above **90%** for 15 min	the machine is overloaded
4	Monitoring → **Uptime** → New check: `https://aimscribe.uiu.ac.bd/health`, from the region nearest Dhaka	**the server stops answering**, and when the certificate nears expiry

Step 13 · Automatic updates from GitHub

From here on the server updates itself. Every two minutes it asks GitHub whether `main` has a new commit. When it has, the server builds the new images while the old ones keep serving, applies any new migration, restarts what changed and waits for `/health/ready`. Healthy: it stays, and the alert webhook says **deployed**. Not healthy: the previous version is put back by itself, and the webhook says **rolled back**. The server **pulls**; GitHub is never given a way in, so the public repository holds no key to the server.

```
cd ~/AIMScribe_v3/deploy/uiu/autodeploy
sudo cp aimscribe-autodeploy.service aimscribe-autodeploy.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now aimscribe-autodeploy.timer
systemctl list-timers aimscribe-autodeploy.timer
```

You should see: one line, with `NEXT` about two minutes away.

13b · Prove it once

On your laptop, change something harmless — a line in a README — commit it and push it to `main`. Then, on the droplet:

```
journalctl -u aimscribe-autodeploy -f          # Ctrl+C to stop watching
```

You should see: `deploying <commit>: <your message>`, the build, `Migrations: 0 applied`, then `deployed <commit>` — usually 5 to 15 minutes, most of it the build — and the same line in the alert channel.

13c · What it will not do

It sees	It does	You do
A commit that is not healthy	Rolls back, alerts, and does not try that commit again	Push a fix; the fix is deployed
A rewritten `main` (a force-push)	Refuses and alerts. A rewritten history is what a hijacked account looks like	Find out why; deploy by hand only if it was you
A file on the droplet edited by hand	Refuses and alerts, rather than overwrite it	`git status` shows it: put it back, or commit the change on GitHub
A new migration	Applies it once, as the tables' owner, and records it in `aims_migrations`	Nothing. A migration is never applied twice
A change in `.env`	Nothing: settings are not in git	Edit `.env`, then `docker compose up -d` by hand

13d · Lock the door it opens from

Whoever can push to `main` can now change the server, so the GitHub account is as important as the droplet's SSH key:

	Do this	Why
1	GitHub → Settings → Password and authentication → **two-factor authentication on**	a stolen password alone cannot push
2	The repository → Settings → Branches → rule for `main`: **block force pushes** and **restrict deletions**	the history cannot be rewritten (the server refuses it anyway)
3	**Optional, strongest:** deploy only commits signed with your key (below)	a pushed commit is not enough; it must carry your signature

```
# On your laptop, once: sign every commit with your SSH key
PS> git config --global gpg.format ssh
PS> git config --global user.signingkey $HOME/.ssh/id_ed25519.pub
PS> git config --global commit.gpgsign true
# On the droplet, as aims: trust that key, and require it
echo "<your email> $(cat <your id_ed25519.pub>)" > ~/.ssh/allowed_signers
git -C ~/AIMScribe_v3 config gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers
sudo mkdir -p /etc/aimscribe
echo AIMS_DEPLOY_REQUIRE_SIGNED=true | sudo tee /etc/aimscribe/autodeploy.env
```

When you want to	Type
Deploy now, without waiting	`sudo systemctl start aimscribe-autodeploy.service`
See what it did	`journalctl -u aimscribe-autodeploy -n 50`
Pause updates (an exam week, a clinic event)	`sudo systemctl stop aimscribe-autodeploy.timer`
Resume them	`sudo systemctl start aimscribe-autodeploy.timer`
See every deploy	`cat ~/.aimscribe-deploy/deployed.log`

PART D

Proving every endpoint

The checks to run after Part C, and to send CMED as evidence

8 · The twelve checks

	Check	Command or action	Pass
1	The name resolves	`nslookup <host> 1.1.1.1`	the droplet's IP
2	The certificate is real	`curl.exe -sI https://<host>/health`	`HTTP/1.1 200`, no TLS error
3	**Health, from outside**	`curl.exe -s https://<host>/health`	`"status":"healthy"`
3b	Every migration is applied (on the droplet)	`docker compose exec -T api curl -s localhost:6000/health`	`"schema":"ok"`, not `"missing ..."`; and `/health/ready` answers 200
4	It is the right system	the same reply	`system` **starts with** `AIMScribe v3`
5	A token is required	POST `/fhir/r4/$process-message` with no token	**401** `INVALID_TOKEN`
6	**API 2 accepts**	POST API 2 with the key	**202** `ACCEPTED`
7	**A repeat is recognised**	POST the identical body again	**200** `ALREADY_RECEIVED`
8	**API 3 Channel B accepts**	POST the prescription	**202** `ACCEPTED`
9	Only 80 and 443 are open	from your laptop: `Test-NetConnection <host> -Port 5432`	`TcpTestSucceeded : False`
10	CMED's door answers, and only with a token	`curl.exe -s https://<host>/fhir/r4/metadata`, then `python tools/channel_b_test.py --server https://<host> --client-id ... --kid ... --key-file ...`	a CapabilityStatement, then **11/11 as expected**
11	The recorders' new endpoints exist and are guarded	`curl.exe -s -X POST https://<host>/api/v2/device/health -H "Content-Type: application/json" -d "{}"`	**401** `device token required` (a 404 means the old code)
12	**A real consultation matches**	one PC, CMED sending all four messages	**confirmed** on the dashboard

Checks 3 to 8 in one command, from a laptop with the repository and Python:

```
PS> python tools\channel_b_test.py --server https://aimscribe.uiu.ac.bd --key <CMED key>
```

Keep the output and send it to CMED: it is the evidence that our side is ready. Its records use test identifiers (`TESTP0001`, `CMED-TEST-01`); they confirm nothing and raise one harmless `unmapped_clinic` alert.

And then the one check that actually matters

Checks 1 to 11 prove the gateway works. **They do not prove the integration works.** Record one real consultation on one clinic PC with CMED sending all four messages, and **confirm that the recording was matched and archived**. Every other check can pass while the five identifying fields silently fail to match — and then the recording is kept with no clinical record attached.

9 · Where things are on the droplet

What	Path	Note
The code and the stack	`/home/aims/AIMScribe_v3/deploy/uiu`	where every `docker compose` command is run
The settings and the keys	`…/deploy/uiu/.env`	**mode 600. Never commit it**
The 16 GB database profile	`…/deploy/uiu/postgres/postgresql.16gb.conf`	part of the code; chosen in 6e
The databases	`/srv/aimscribe/db`	grows 20–40 GB a year
The archive	`/srv/aimscribe/archive`	**clinic / doctor / date / consultation**. Each WAV leaves this disk 2 days after its cloud copy is proven (6e); its JSON and manifest stay
Nightly backups	`/srv/aimscribe/backup`	encrypted, 30 days
The permanent cloud copy	R2 bucket `aimscribe-archive`	encrypted with `AIMS_COPY_KEY`

PART E

Daily and weekly checks

Five minutes a day keeps a silent failure from becoming a lost month

10 · Every clinic day (5 minutes)

```
ssh aims@<droplet-ip>
cd ~/AIMScribe_v3/deploy/uiu
A=$(grep ^AIMS_ADMIN_KEY= .env | cut -d= -f2); H=http://127.0.0.1:6001/api/v2

docker compose ps                    # nine Up, four (healthy)
df -h /srv | tail -1                       # Use% below 80
curl -s $H/dashboard/summary -H "X-Admin-Key: $A" | jq .
curl -s "$H/admin/alerts?limit=20" -H "X-Admin-Key: $A" | jq .
```

Look at	Healthy	Act when
Services	nine Up	**anything restarting or exited**
Disk	under 80%	**over 80%**: archived WAVs leave 2 days after their copy is proven, so a rising disk means copies are not being proven — look for `cloud_copy_overdue` in the alerts
Unconfirmed recordings	zero once any outage is over	**any left**: kept and archived, but CMED's record never arrived or did not match. Ask CMED to check its outbox for that visit. Nothing is ever deleted for this
Offline periods	dashboard → Recorders → *Offline this week*	a room offline for hours, or *microphone silent at times*: look at that PC's network or microphone
Silent recorders	every clinic PC reported recently	a PC silent during clinic hours: killed agent or no network
Alerts	nothing new	`unmapped_clinic`, `chain_entry_rejected`, `spool_critical`, `authorisation_held`, `start_abandoned`, `reassign_not_applied`: investigate the same day. `possibly_two_consultations`: check the recording and split it if two patients ran together

11 · Every week (20 minutes)

	Check	How
1	Last night's backup exists	`ls -lt /srv/aimscribe/backup | head -3`
2	The backups reached R2	`docker compose run --rm backup list-offsite`: last night's two files are there
3	Cloud copies are arriving	Cloudflare → R2 → `aimscribe-archive`: object count grew
4	Security updates applied	`cat /var/run/reboot-required` — if it exists, reboot **after clinic hours**
5	The certificate	Uptime check green; nothing from Let's Encrypt in the owner mailbox
6	Costs	DigitalOcean and Cloudflare billing pages, against the plan

12 · Every month

The restore drill (step 12b). Then write the date and the row counts in the access sheet. If it fails, it is the most important problem on the list until it is fixed.

13 · The dashboard page

From your laptop, through an SSH tunnel (the internet gets `403` for this page unless your address is in `AIMS_ADMIN_ALLOW`):

```
PS> ssh -N -L 6001:127.0.0.1:6001 aims@<droplet-ip>     # leave it open
#   then open  http://localhost:6001/api/v2/dashboard/  in the browser
```

The dashboard asks for the admin key, which it keeps in memory for that tab only. The gateway gives this one path a policy that allows the page's own script and style files and its own API calls, and nothing else; every other path stays closed to browsers. The JSON calls in section 10 carry the same numbers.

PART F

Updates, rollback, what was fixed, go-live

What to do later, and what to settle before the clinics depend on it

14 · Updating to a new release

Updates arrive by themselves (step 13): push to `main` and the server deploys it. This section is for the rare update by hand, and for a server built before the migration ledger.

Migrations are applied once each, automatically. Every update runs `deploy/uiu/postgres/migrate.sh`: each new file in `backend/scripts` is applied in order, as the role that owns the tables, and recorded in the `aims_migrations` table. A file already recorded is never applied again — re-running an old migration on a database full of recordings is not safe — and one that was edited after it was applied is reported, not re-run: a change to the schema is always a new file. By hand:

```
cd ~/AIMScribe_v3/deploy/uiu
docker compose exec -T postgres sh -s < postgres/migrate.sh
```

You should see: `Migrations: 0 applied, 25 already in place, 0 changed since applied.` The numbers grow with each release; `changed` stays 0.

A server built before 24 October 2026 has no ledger, and the runner refuses it rather than guess which migrations it has. Adopt it once: `/health` (step 9, row 3b) names the first migration it is missing — adopt through the one before. If it names none, adopt through the last file in `backend/scripts`.

```
# e.g. /health says 023 is missing:
docker compose exec -T -e AIMS_MIGRATIONS_ADOPT_THROUGH=022 \
  postgres sh -s < postgres/migrate.sh
```

You should see: the older files listed as `adopted (already on this server)`, then `applying 023_...` and every later one.

A server set up before 018: give the API its own login. Until 018 the API logged in as `aims_recordings`, the owner of every table — which could switch off the audit log's protection. From 018 it logs in as `aims_api`, which reads and writes rows and nothing more, and the owner has no login at all. Once, on the droplet:

```
cd ~/AIMScribe_v3/deploy/uiu
setv() { sed -i "s|^$1=.*|$1=$2|" .env; grep -q "^$1=" .env || echo "$1=$2" >> .env; }
setv AIMS_API_DB_PASSWORD "$(openssl rand -hex 32)"
U=$(docker compose exec -T postgres printenv POSTGRES_USER)
P=$(grep ^AIMS_API_DB_PASSWORD= .env | cut -d= -f2)
docker compose exec -T postgres psql -v ON_ERROR_STOP=1 -U "$U" -d postgres \
  -c "CREATE ROLE aims_api LOGIN PASSWORD '$P'"
unset P
# a server this old has 016 at most: adopt it, then 017 onwards are applied
docker compose exec -T -e AIMS_MIGRATIONS_ADOPT_THROUGH=016 \
  postgres sh -s < postgres/migrate.sh
docker compose exec -T postgres psql -U "$U" -d postgres \
  -c "ALTER ROLE aims_recordings NOLOGIN"
# the internal network gained a fixed range: recreate the networks once
docker compose down && docker compose up -d --build
```

You should see: `CREATE ROLE`, the migrations without an error, `ALTER ROLE`, and every service back `Up`. `/health` says `"database":"connected"` - now as `aims_api`. If the API log says `password authentication failed for user "aims_api"`, the password in `.env` and the role differ: set it again with `ALTER ROLE aims_api PASSWORD ...` the same way.

Order: the server first, then the recorders. A new recorder sends each patient click and prescription signal to `/session/marker`, and its offline log; an old server answers 404, and a new one without 015 answers 500. The recorder keeps them waiting until the server is ready — nothing is lost — but nothing after that step in the recording uploads until then. So update the server first — its migrations come with it — then roll the installer out to the clinic PCs. **Upgrade the archive worker together with the server:** the server now receipts only the pieces the worker says it joined, so an old worker gets 409 and archives nothing until it is upgraded. **Once every PC runs 3.6.1** (`v_devices`, or the dashboard's Recorders panel), set `AIMS_REQUIRE_DEVICE_SIGNATURES=on`, `AIMS_REQUIRE_SIGNED_REFUSAL=on` and `AIMS_MIN_RECORDER_VERSION=3.6.1` in `.env` and `docker`

`compose up -d api`; before that they would refuse the 3.4 and 3.5 PCs. Every request then carries an `X-Request-ID` that the gateway sets; quote it when reading the API log.

15 · If it has to be undone

	Situation	What to do
1	A release misbehaves	If it is not healthy the server has already rolled it back (step 13). If it is healthy but wrong: on your laptop `git revert <commit>` and push — the revert is a new commit, and it is deployed like any other. **Never force-push**: the server refuses a rewritten `main`
2	The deployment fails before any clinic uses it	**Nothing to undo.** Destroy the droplet. The old stack is untouched
3	It works, but a problem appears on day one	Re-run the installer on the PCs with the old address and fresh tokens from the old server, and tell CMED to revert their setting. **Keep the droplet running** — it holds recordings the old one does not
4	The droplet is lost entirely	Restore DigitalOcean's weekly backup as a new droplet, or rebuild with this manual and restore last night's database backup. **The cloud copies in R2 are unaffected** — if `AIMS_COPY_KEY` was kept off the machine

16 · What was found, and what is fixed

Found while checking this manual against the code on 15 October 2026, and fixed in the code the same week. Listed so that anyone who read an earlier copy knows the workaround is gone.

	Was	Now
1	Every archived WAV stayed on the server disk: full in about 9 days	Removed from this disk `AIMS_ARCHIVE_LOCAL_RETENTION_DAYS` days (0 here) after its cloud copy is proven, and never without one. JSON, manifest and catalogue stay; the manifest says where the copy is
2	The backup service never made a backup, and could not encrypt	Its own image with `openssl`; the schedule is plain arithmetic. Step 12, run with `--now` to prove it
3	The cloud copy was FLAC	**The raw WAV**, as agreed, encrypted, and decrypted again and hashed before upload. Older FLAC copies still restore
4	The dashboard loaded blank through the gateway	A policy for that path only; checked in a browser behind Caddy
5	`/health` said storage was connected without asking	It asks both buckets, every 30 seconds at most, and says which one failed
6	The monitor's 500 GB floor alarmed for ever on a 320 GB disk; two floors never reached it	Each floor capped at 15% of its disk; all three passed through
7	The gateway did not start: its log filter named a module Caddy does not have	Fixed, and the whole Caddyfile validated in `caddy:2.8-alpine`
8	The repository's public keys did not match the server	11b now proves the pair before the installer is built
9	A recording without CMED's API 2 was deleted after 24 hours, then kept for ever	**Kept until CMED's server confirms it** (API 2 or API 3, however late); erased 7 days after it was made if CMED never does — never while CMED is silent for that clinic, never under a legal hold (17d)
10	A server "no" — a wrong key, an unmapped clinic — deleted the recording on the laptop	**Held on the laptop** and asked again every 5 minutes; the fix releases it
11	A wrong patient opened first locked the room until Stop	The prescription decides: the whole recording moves to that patient (013)

	Was	Now
12	An offline laptop's heartbeats and alerts were lost	Logged on the laptop, delivered on reconnect, then deleted there (014)

17 · Who may send audio

Anyone on the internet can reach the server's address. **No one but an enrolled clinic PC can store audio in it**, and nothing reaches the dataset until CMED's own server has vouched for the visit:

	Layer	What it stops
1	**Every request is signed by the PC's own key, held in its TPM** (recorder 3.6; older PCs: 17b). Each request carries the PC's token, a timestamp, a one-time nonce and a signature over all of it (`device_nonces` remembers the nonces for 15 minutes; `v_device_requests` shows them by PC and clinic)	A copied token alone is useless; a recorded request cannot be replayed
2	**Joining needs a one-time token, or CMED.** A token is bound to one clinic, expiring, kept only as a hash. Without one (3.7), a PC with a TPM key is **pending** until CMED's server confirms two of its recordings (17c)	An outsider cannot become a trusted PC
3	**Rate limits**: 1,200 requests a minute per address; 100 failures in 5 minutes shuts an address out for 15; 10 for one credential	Guessing, and flooding the API
4	**A PC records for its own clinic only.** Anything else is filed under its clinic and raised as `clinic_mismatch`	One clinic's PC posing as another's
5	**CMED confirms each visit** from its own server, signed with CMED's key (Channel B). Unconfirmed audio is kept, but never enters the dataset	Audio from a stolen or rogue PC passing as a real consultation
6	**Revocation** in one call; `hold` quarantines what it had not finished	A lost or stolen PC
7	**Admin and dashboard** answer only on the droplet itself (an SSH tunnel) or listed addresses	Anyone else administering the server

17b · Set at go-live

Setting	Value	Why
`AIMS_REQUIRE_DEVICE_SIGNATURES`	`on`	Every PC must sign every request. Today a PC with a software key (recorder 3.4) only has to sign once it has signed; `v_devices` shows each PC's key kind
`AIMS_MIN_RECORDER_VERSION`	`3.6.1`	Older recorders are refused and told to update
`AIMS_REQUIRE_SIGNED_REFUSAL`	`on`	A patient's refusal must be signed by the PC that recorded
Enrolment tokens	24–72 hours	Mint them on the day of the install, not two weeks ahead. Until it is used, the token sits in a file on the PC
Old PCs	revoked	A PC replaced or reinstalled: revoke the old entry. `v_devices` lists every PC with its state
`AIMS_UNCONFIRMED_ERASE_DAYS`	`7`	Days a recording waits for CMED before it is erased (17d)
`AIMS_SELF_REGISTRATION`	`on`	`off`: every PC needs a token again

17c · Enrolment without a token (server 025, recorder 3.7)

A clinic PC needs no enrolment token. It can ask to join, and its audio reaches the server at once, but nothing it records is kept unless CMED's own server confirms that visit, and it becomes a trusted PC only when CMED has confirmed two of them. The dashboard's PC table shows a joining PC as **Waiting for CMED**, with Approve and Reject.

	What happens	Why an outsider gets nowhere
1	The installer makes the PC's key inside its TPM, as now. No token	The key never leaves the chip

	What happens	Why an outsider gets nowhere
2	The first time CMED opens a patient on that PC, the recorder registers: its public key, proof it holds the key, and the clinic code from CMED's own trigger. The server files it as **pending** for that clinic	A pending PC uploads at most `AIMS_PENDING_UPLOAD_MB` (500 MB), and none of it is kept unless CMED confirms that visit
3	CMED's server sends API 2 for that visit, signed with CMED's key. When it matches what the pending PC asked to record — patient, doctor, clinic, date and time — on two visits, the PC is **approved by itself** and its held audio uploads	Approval needs CMED's server, which an outsider does not have, to confirm visits the PC really recorded
4	The dashboard lists every pending PC (name, address, clinic): approve or reject at once, without waiting	A person can always overrule
5	Limits: 20 pending PCs a clinic, a pending PC expires after 7 days, registrations rate-limited per address	Registering thousands of fake PCs

Further, optional: TPM key attestation — Windows can produce a statement, chained to the TPM maker's certificate, that the key was made inside a real TPM. It proves the hardware, not just the key, and is not built yet.
Order: update the server first, then install 3.7 on new PCs; an older recorder still enrols with a token.

17d · Only what CMED confirms is kept

	What happens
Audio	Reaches the server at once, whatever CMED does: a late API 2 or API 3 costs nothing
Confirmed	API 2 or API 3 from CMED's server names the visit, to the second — minutes or days later. Kept, copied to the archive, in the dataset
Waiting	Archived on the server, not copied. `sessions.erase_after` says when it goes; `v_audio_problems` lists those due within 48 hours, and an alert counts them per clinic
Never confirmed	Erased `AIMS_UNCONFIRMED_ERASE_DAYS` (7) days after it was made, as a refused one is: audio, files and clinical record go; a row without the patient stays
CMED silent	A clinic CMED has sent nothing for in 24 hours erases nothing, however old: an outage is not a reason to lose consultations. An alert says so
Rescue one	A legal hold, or the admin confirm (a name and a reason), before its date

18 · Go-live checklist

	Item	Where
■	Both accounts owned by your-team-email@example.org, two-factor on, recovery codes stored	Part 0
■	Billing alert set on DigitalOcean	0.2
■	Three R2 buckets, three tokens, public access off; bucket lock on the archive	6, 6b
■	Droplet in BLR1, `s-8vcpu-16gb`, backups and monitoring on	step 1
■	Name resolves to the droplet	step 2
■	Firewall: 22 from your IP only; 80 and 443	step 3
■	Root and password logins off; timezone Asia/Dhaka; swap on	step 4
■	Every `ok` in 6g; `compose settings: OK`	step 6
■	**Six secrets stored off the machine**	6g
■	**Key fingerprints identical** on the droplet, the laptop and the first PC; installer rebuilt	6d, 11b
■	Archive folder owned by 10001	step 7
■	Nine services up; certificate obtained	step 8
■	`/health` healthy **from outside**	step 9
■	**All seven clinics registered with CMED's codes**	step 10
■	CMED key issued and delivered; CMED has the host	step 10, Part A
■	One PC installed and a real consultation **confirmed**	step 11, Part D
■	First backup made with `--now`, and **restored**	step 12
■	Email alerts and the uptime check active; webhook set	12c, 6f
■	`/health` on the droplet shows both buckets `connected`	step 9
■	`migrate.sh` reports `0 changed since applied` (a new server has the ledger; an old one is adopted once)	14
■	Checks 10 and 11 answer 401, not 404	Part D
■	channel_b_test output sent to CMED	Part D
■	**Automatic updates on**: timer enabled, one test commit deployed and announced	step 13
■	GitHub: two-factor on; `main` protected from force-push and deletion	13d
■	Device signatures required; minimum recorder 3.6.1; old PCs revoked	17b
■	One cloud copy downloaded, restored and played	6b, 12b

19 · One page to remember

Question	Answer
Which compose file?	`deploy/uiu/docker-compose.yml` — the server stack, despite the folder name
Which droplet?	`s-8vcpu-16gb`**, Bangalore**, Ubuntu 24.04, backups and monitoring on
What must exist first?	**A domain name pointing at the droplet**
Which ports open?	**80 and 443 only**, plus SSH from your own IP
How many buckets?	**Three**: `aimscribe-segments`, `aimscribe-archive` and `aimscribe-backups`
How do updates arrive?	**Push to** `main`. The droplet deploys it within minutes, checks it, and rolls back by itself. Step 13
What must change for 16 GB?	`AIMS_PG_CONF`, `AIMS_PG_MEMORY`, `AIMS_ARCHIVE_MEMORY`, `AIMS_WORK_TMPFS`
Where do CMED's clinic codes go?	`cmed_hospital_id` on each clinic. Step 10
Which secrets must leave the machine?	**Six**, led by `AIMS_COPY_KEY` and `AIMS_BACKUP_KEY`
What does CMED change?	**One setting** — our host. Their page changes nothing
What does each clinic PC need?	**One installer run** with the new address and its own token
First thing after it is up?	`curl https://<host>/health` **from somewhere else**
Most valuable half hour?	**The restore drill.** Step 12b
Who gets the notices?	your-team-email@example.org

Prepared by AIMS LAB, United International University. Companions: CMED_1 Software Integration, CMED_2 Server Deployment, and the Cloud Hosting Plan. Every port, path, service, setting and command here was checked against the stack itself.